AI Engineering and Claude Architect · Part V, Claude Code

Claude Code, Exercises

Six Worked Exercises

Choose tools, sort a messy configuration, write a safe hook, plan a migration, build a CI review, and defend a decision.

Module	AI12 of 13
Covers	All 5 lessons
Time	~3 hours. Exercises 2 and 3 can be done on a real repository

Exercise 1: Pick the tool

Name the tool or sequence for each, under the exam's framing. **(a)** Find every file named `*Controller.java`. **(b)** Find where `MAX_RETRIES` is set. **(c)** Change one of 9 identical `timeout: 30` lines. **(d)** Rename a variable throughout one file. **(e)** Find all callers of `charge`, which is re-exported as `pay` and `bill`. **(f)** Learn how login works in an unfamiliar repo.

Solution

(a) Glob, `**/*Controller.java`. **(b)** Grep for `MAX_RETRIES`. **(c)** Edit fails on a non-unique match, so the guide's answer is Read then Write. In practice, a longer `old_string` including its surrounding block is smaller and safer. **(d)** Edit with `replace_all: true`, per the tools reference. **(e)** Identify all exported names (`charge`, `pay`, `bill`), then Grep for each. **(f)** Grep for an entry point such as `login`, Read it, follow imports, and delegate wide searches to a subagent, keeping notes.

Exercise 2: Sort this configuration

A team's `CLAUDE.md` is 700 lines and contains: the build commands; a 60-step release procedure; React conventions; SQL migration rules that apply in four directories; "NEVER commit to main" in capitals; a developer's preference for terse replies; a GitHub token; and "write clean code."

Place each item, and say why.

Solution

Item	Place	Why
Build commands	Root `CLAUDE.md`	Needed everywhere, and Claude can't guess them
Release procedure	Skill, perhaps with `disable-model-invocation`	Occasional, and long
React conventions	Rule with `paths: ["**/*.tsx"]`	Loads only when relevant
Migration rules	Rule with a glob matching all four	One file, spread-out targets
Never commit to main	Hook, or better, branch protection	A guarantee can't be a line of text
Terse replies	`~/.claude/CLAUDE.md`	Personal
GitHub token	Environment, with `${GITHUB_TOKEN}` in `.mcp.json`. **Rotate it now**	It's been committed
"Write clean code"	Delete	Unverifiable, and costs attention

Then check with `/memory` that the rules load as expected.

Exercise 3: Write a safe hook

Write the logic of a `PreToolUse` hook that protects `infra/prod/`. **(a)** What should it match? **(b)** What does it do for each outcome? **(c)** Name three cases to test. **(d)** What can't it stop, and what covers that?

Solution

(a) `Edit|Write`, and also `Bash`, because a shell command can change those files.

(b) Parse the JSON from stdin. If a file path resolves into `infra/prod/`, or a command mentions it, write a reason to stderr and exit 2. Otherwise exit 0. Wrap everything so that an unexpected error also exits 2 with a reason, because other non-zero exits let the action proceed on most events.

(c) An edit inside the directory is blocked, with a useful message. An edit elsewhere proceeds. Malformed JSON is blocked. Also test a relative path with `..`, and a Bash command mentioning the path.

(d) Text checks can't catch every form of a command: a variable holding the path, a script that writes there, or a wildcard. Sandbox filesystem rules or deny rules cover that independently of command text, and file permissions on the machine are stronger still. And a hook doesn't review the content of permitted changes.

Exercise 4: Plan a migration

You must move a service from one HTTP client library to another. It has about 80 call sites, some with custom retry behavior, and a mix of sync and async.

(a) Plan or direct? **(b)** Lay out the workflow, naming the tools. **(c)** How do you specify the conversion? **(d)** What's your session strategy if it spans days?

Solution

(a) Plan mode. It's large, it has variations, and being wrong is costly.

(b) Use Explore to find call sites and group them by pattern, returning a summary. In plan mode, decide how each group converts, and how retries map. Write the plan to a file. Execute by group, running tests after each one, and commit each group. Finally, a fresh session reviews the diff against the plan.

(c) With 2–3 before-and-after examples per pattern, including a custom retry. Write tests for retry behavior first, then iterate on failures. Retry and timeout behavior interact, so describe those together.

(d) Name the session and use `--resume` while it's valid. Keep the plan and progress in a file, and update it as you go. If a rebase makes most results stale, start a new session from that file, rather than resuming. To compare two retry designs, fork from the analysis, in separate worktrees, as forks don't separate files.

Exercise 5: Build the CI review

Design a pipeline for a public repository that reviews PRs, including those from forks, and posts inline comments. Give: **(a)** the core command; **(b)** what the wrapper checks; **(c)** how you handle a new commit; **(d)** how you handle fork PRs; **(e)** how you'd roll it out.

Solution

(a) `gh pr diff "$PR" | claude -p "<criteria and prior findings>" --output-format json --json-schema "$SCHEMA" --max-turns 5 --allowedTools "Read"`. Run it in an instance independent of whatever wrote the code. Use `--bare` with explicit context for reproducibility, if your version supports it.

(b) Exit code zero, and `structured_output` present. Otherwise the job reports "review didn't run," and doesn't pass. Parse with a real JSON library. Also check for skipped MCP servers, if any are used.

(c) Fetch earlier findings, pass them in, and ask for only new or unaddressed issues. In code, compare fingerprints of file, rule and normalized snippet, with no line numbers, so that moved issues don't repeat. Record accepted findings.

(d) Don't run with secrets on fork code. GitHub withholds secrets from fork-triggered runs anyway, so reviews there need a design decision: for example, a maintainer applies a label after reading the PR, and a separate trusted job reviews only the diff as data, without checking out and running the fork's configuration, as `-p` would run its hooks with no prompt. Read criteria from the base branch, and keep tools read-only.

(e) Start as advisory. Measure dismissals by `detected_pattern` and category. Disable noisy categories, improve their criteria with examples, then let proven categories block merges. Decide what happens during a provider outage: a required check could block everything, so have an override.

Exercise 6: Defend a decision

The decision: a platform lead wants all 200 developers on one enforced Claude Code setup: a 1,000-line managed CLAUDE.md, every workflow as a hook, and personal skills banned. Write your recommendation.

Model answer

I'd enforce a small set of real rules, and keep the rest as guidance.

What's right about the proposal. Consistency across 200 people has value. Security rules shouldn't depend on each developer's configuration. Managed settings exist for this, and developers can't override them.

What's wrong with 1,000 lines. Everything in a CLAUDE.md costs attention on every task, the docs target under 200 lines per file, and longer files reduce adherence. And since it's context, it wouldn't be enforced anyway. The size buys neither compliance nor quality. I'd keep a short managed file for what truly applies everywhere, and move conventions to project-level path rules owned by each team.

What's wrong with every workflow as a hook. Hooks suit things that must always or never happen. The docs say explicitly that hooks for mere preferences should be deleted, as models judge exceptions better from the code in front of them. Each hook is code to maintain, adds latency, and can fail open if it's written badly. I'd keep a short list: no secrets read, no edits to generated or production config, and the license header. Everything else stays in skills.

What's wrong with banning personal skills. They're in `~/.claude/skills/`, and they affect only their owner. The harm is low, and they're how good practice is discovered. Good ones can be promoted to the team. What matters is what can touch shared systems, and that's controlled by deny rules, sandboxing and MCP allowlists.

What I'd enforce. Managed deny rules, sandbox settings, an MCP allowlist, no bypass mode, and CI as the last word: branch protection and independent review.

Risks I'm accepting. Some inconsistency between teams, and people will sometimes ignore guidance. CI and code review catch what matters. I'd measure review findings and incidents, rather than assuming.

What would change my mind. A regulated environment with audit requirements on process itself could justify stricter controls, and I'd still implement them as enforcement rather than as text. Also, evidence that a particular practice failing repeatedly causes incidents would make it a candidate for a hook.